PixCrawler

Project Proposal

Image Dataset Generation SaaS Platform

Automated Discovery, Download, and Generating
of Images Datasets with Cloud-Native Architecture

Development Team

Amr Muhamed Fathy
Marwan Hatem Ahmed
Aya Alaa Hamed
Mohamed Hassan Abdelmonem
Osama Mohamed Ali Abdel-Gaffar
Ganna Tarek Mohamed Mansour

November 03, 2025

Contents

1 Executive Summary **2**
1.1 Platform Vision . . . 2

2 Problem Statement **2**

3 Project Objectives **2**
3.1 Core Objectives . . . 2

4 Scope **4**
4.1 In-Scope Work . . . 4
4.2 Out-of-Scope . . . 4

5 Architecture Overview **4**
5.1 Key Architectural Components . . . 5

6 Technical Stack **5**
6.1 Technology Overview . . . 5

7 Functional Requirements **6**

8 Non-Functional Requirements **6**

9 Validation Factory System (Internal Overview) **7**
9.1 Layer 1: Basic Validation (Fast) . . . 7
9.2 Layer 2: Standard Validation (Moderate) . . . 7
9.3 Layer 3: Deep Validation (Comprehensive) . . . 7

10 Deliverables **7**

11 Project Plan & Milestones **8**

12 Risks & Mitigation **9**
12.1 Critical Risk Analysis . . . 9

13 Assumptions & Dependencies **9**
13.1 Technical Assumptions . . . 9
13.2 External Dependencies . . . 9

14 Future Opportunities **9**
14.1 Enhancement Roadmap . . . 9

15 References **10**

1 Executive Summary

PixCrawler has evolved into a **cloud-native SaaS platform** that automates the discovery, download, organization, and validation of images from multiple sources. Built on a modern microservices architecture, PixCrawler provides researchers, data scientists, and ML engineers with a scalable, distributed solution for creating high-quality image datasets.

1.1 Platform Vision

This document describes the vision, objectives, scope, architecture, and delivery plan for PixCrawler 2.0—a fundamental reimagining of the original tool as a production-ready, enterprise-grade SaaS platform with SDK integration capabilities.

Key Transformation

PixCrawler has transitioned from a CLI-based tool to a full-featured SaaS application with:

- Web-based interface for dataset configuration and monitoring
- Python SDK for programmatic integration
- Distributed task processing with Celery
- Multi-layered validation factory system
- Cloud-native deployment on Azure

2 Problem Statement

Creating large, clean image datasets remains time-consuming and complex. Traditional approaches suffer from:

- **Slow, sequential workflows** that don't scale with dataset size
- **Limited validation options** resulting in poor data quality
- **Lack of infrastructure** for distributed, parallel processing
- **No programmatic access** for integration into ML pipelines
- **Insufficient monitoring** of long-running dataset builds

PixCrawler 2.0 addresses these challenges with a distributed, task-based architecture that can be accessed via web interface or SDK, providing flexibility for both interactive and automated workflows.

3 Project Objectives

3.1 Core Objectives

The following objectives drive the development and design decisions for PixCrawler 2.0:

1. **SaaS-First Architecture** — Provide a web-based platform with account management, job monitoring, and dataset storage
2. **SDK Integration** — Offer a Python SDK (Python 3.8+) enabling programmatic access to all platform features
3. **Flexible Data Acquisition** — Support multiple scraping strategies beyond search engines, with modular source adapters
4. **Intelligent Validation Factory** — Implement three-tiered validation system where users configure validation depth based on quality vs. speed requirements
5. **Distributed Processing** — Leverage Celery and Redis for scalable, fault-tolerant task execution
6. **Smart Keyword Generation** — Continue leveraging lightweight LLMs (gpt4-mini via g4f) for search term enrichment
7. **Automated Labelling** — Produce labels in multiple formats (TXT, JSON, CSV, YAML) matching dataset structure
8. **Cloud-Native Deployment** — Deploy on Azure App Service with Azure Monitor for observability
9. **Type Safety & Code Quality** — Maintain 100% type-hinting coverage with rigorous static analysis

4 Scope

4.1 In-Scope Work

Area	Description
Image Acquisition	Multiple strategies: multi-engine search (Google, Bing, Baidu, DuckDuckGo), direct URL lists, API integrations, custom scrapers
Configuration	YAML-based config (primary) with legacy JSON schema support
Validation Factory	Three-tier validation system: • Layer 1: Basic integrity (fast) • Layer 2: Format validation + hash deduplication (moderate) • Layer 3: Perceptual hashing + deep analysis (slow)
Task Processing	Celery-based distributed task queue with Redis backend
Web Application	FastAPI backend with RESTful API and WebSocket support
Frontend	Modern web UI for dataset configuration, job monitoring, and management
SDK	Python package (3.8+) for programmatic platform access
Keyword Engine	LLM-powered term generation/augmentation using g4f proxy
Labelling	Multi-format label writers mirroring directory structure
Reporting	Real-time job status, detailed statistics, failure analysis
Monorepo Structure	Organized packages: backend, frontend, builder, validator, celery-core, sdk, shared utilities
Cloud Deployment	Azure App Service, Azure Monitor, with future support for Azure Redis, Azure Functions, Azure Container Instances

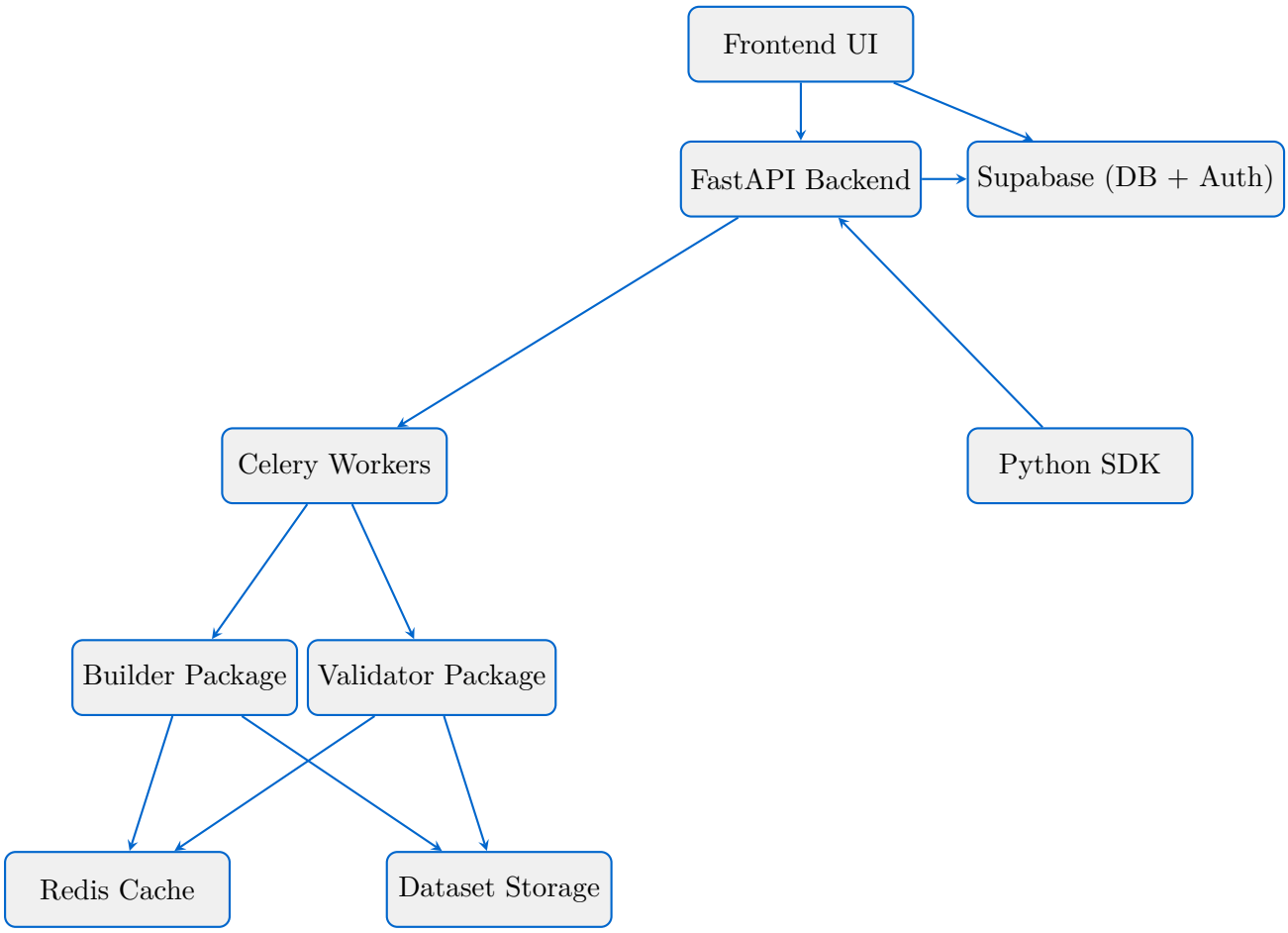
4.2 Out-of-Scope

The following features are explicitly excluded from the initial release:

- On-device image augmentation (rotation, flips, color-jitter)
- Mobile native applications (iOS/Android)
- Paid/commercial image APIs (e.g., Shutterstock, Getty) as built-in sources
- Video or non-image media processing
- Built-in model training capabilities

5 Architecture Overview

PixCrawler 2.0 employs a microservices-inspired monorepo architecture with clear separation of concerns:



5.1 Key Architectural Components

- **Frontend**: Modern web interface for interactive dataset creation and monitoring
- **FastAPI Backend**: RESTful API server handling authentication, job orchestration, and WebSocket connections
- **Celery Workers**: Distributed task processors for parallel image acquisition and validation
- **Redis**: Message broker and caching layer for Celery tasks
- **Builder Package**: Core logic for image discovery, download, and organization
- **Validator Package**: Three-tiered validation factory implementation
- **Python SDK**: Client library exposing platform functionality for external integration

6 Technical Stack

6.1 Technology Overview

PixCrawler 2.0 leverages a modern Python ecosystem and cloud-native architecture for performance, scalability, and maintainability:

Category	Technologies
Source Language	Python 3.11+ (backend/core packages)
SDK Support	Python 3.8+
Backend Framework	FastAPI with Pydantic v2
Task Queue	Celery 5.x with Redis broker
Database & Authentication	Supabase (PostgreSQL + Auth)
Core Libraries	requests, icrawler, duckduckgo-search, pillow, imagehash, tqdm, pyyaml, jsonschema, g4f
Type Checking	MyPy with strict mode, 100% coverage
Code Quality	Ruff (linting and formatting)
Testing	PyTest, pytest-asyncio, pytest-cov
Packaging & Dependencies	uv (fast dependency resolver and package manager)
Cloud Platform	Azure App Service, Azure Monitor
Future Cloud	Azure Redis Cache, Azure Functions, Azure Container Instances
Frontend	Next.js with Shadcn/UI components
OS Support	Windows, macOS, Linux

7 Functional Requirements

ID	Requirement
FR-1	User shall authenticate via web interface or SDK with API key
FR-2	User shall configure datasets via YAML or legacy JSON format
FR-3	System shall support multiple image acquisition strategies (multi-engine, URL lists, custom scrapers)
FR-4	System shall distribute tasks across Celery workers for parallel processing
FR-5	System shall apply user-selected validation layers (1-3) during dataset building
FR-6	System shall cache progress in Redis and resume interrupted jobs
FR-7	System shall organize images in `datasets/<name>/<category>/<source>/` structure
FR-8	System shall generate labels in formats specified by user (TXT, JSON, CSV, YAML)
FR-9	System shall provide real-time job status via WebSocket and REST API
FR-10	SDK shall expose all platform features with type-safe interfaces
FR-11	System shall log all operations and generate comprehensive job reports

8 Non-Functional Requirements

- **Performance**: Support 500+ concurrent image downloads with Celery workers; sub-second API response times
- **Scalability**: Horizontal scaling via additional Celery workers; handle 100+ simultaneous jobs

- **Reliability**: <0.5% failure rate after retries; automatic job recovery from Redis state
- **Maintainability**: 100% type-hinting coverage; comprehensive docstrings; 90%+ test coverage
- **Security**: JWT authentication; API key management; input sanitization; rate limiting
- **Observability**: Azure Monitor integration; structured logging; distributed tracing
- **Portability**: Containerized deployments; environment-agnostic configuration

9 Validation Factory System (Internal Overview)

PixCrawler 2.0 internally incorporates a multi-layered validation factory concept to support varying levels of data quality assurance. This modular design allows the team to switch between lightweight and intensive validation modes based on internal priorities and experiment goals.

9.1 Layer 1: Basic Validation (Fast)

- Focused on lightweight prechecks and structural consistency
- Designed to ensure minimal input reliability for rapid iteration
- **Hint:** Ideal for early-stage crawls or internal testing

9.2 Layer 2: Standard Validation (Moderate)

- Expands checks to cover format-specific integrity and consistency
- Balances verification depth with acceptable processing time
- **Hint:** Recommended for internal pipelines targeting stable data

9.3 Layer 3: Deep Validation (Comprehensive)

- Intended for advanced or high-stakes verification experiments
- May integrate perceptual or content-level analysis components
- **Hint:** Used internally for high-quality benchmarks and validation studies

Note: These layers are configurable components within the internal pipeline, not publicly exposed in the SDK or user-facing API. They serve as internal mechanisms for experimentation and quality assurance during development.

10 Deliverables

1. Source code monorepo (GitHub) with MIT license
2. **SaaS Platform**: Deployed web application on Azure App Service
3. **Python SDK**: Published to PyPI as `pixcrawler-sdk` (Python 3.8+)
4. **API Documentation**: OpenAPI/Swagger specification
5. **SDK Documentation**: Comprehensive API reference and usage examples
6. Monitoring dashboards and alerting configuration

11 Project Plan & Milestones

Week	Milestone
1-2	Establish monorepo structure; set up FastAPI backend foundation; implement authentication
3-4	Integrate Celery with Redis; configure task distribution and monitoring
5-6	Implement Builder package with multiple acquisition strategies
7-8	Develop and integrate three-tier Validation Factory
9-10	Build frontend web application; implement WebSocket real-time updates
11-12	Conduct integration between frontend and backend; finalize dataset workflow pipeline
13-14	Azure deployment and cloud monitoring setup (App Service, Azure Monitor)
15	Develop Python SDK (`pixcrawler-sdk`); integrate with backend API
16	Testing phase: integration, performance, and load testing
17	Documentation finalization (API Docs, SDK Docs, User Guide)
18	v2.0 Beta and production launch

12 Risks & Mitigation

12.1 Critical Risk Analysis

Risk	Impact	Likelihood	Mitigation
Search engine blocking	High	Medium	Multiple acquisition strategies; rotate user-agents; respect robots.txt; implement exponential backoff
Azure service disruptions	High	Low	Multi-region deployment preparation; comprehensive error handling; local fallback modes
Celery worker failures	Medium	Medium	Automatic task retry; worker health monitoring; graceful degradation
LLM API instability	Medium	Medium	Abstracted keyword module; fallback to rule-based generation
Validation performance bottlenecks	Medium	High	Tiered approach allows users to choose; optimize Layer 3 with batch processing
SDK version compatibility	Low	Medium	Support Python 3.8+; comprehensive testing matrix; semantic versioning

13 Assumptions & Dependencies

13.1 Technical Assumptions

- Azure services remain available with current SLA guarantees
- Free public search endpoints continue to be accessible
- g4f maintains stable interface to LLM providers
- Redis can handle projected message throughput
- Users have sufficient storage quota for datasets

13.2 External Dependencies

- Third-party Python packages maintain compatibility
- Search engine terms of service remain permissive for research use
- Azure pricing remains within budget constraints
- SSL/TLS certificates for secure platform access

14 Future Opportunities

14.1 Enhancement Roadmap

The following features represent potential future enhancements beyond v2.0:

1. **Advanced Cloud Integration**

- Migration to Azure Redis Cache for improved performance
- Azure Functions for serverless task processing
- Azure Container Instances for isolated job environments

2. **Enhanced Validation**
   - CLIP-based semantic similarity detection
   - Automatic NSFW/inappropriate content filtering
   - Custom validation plugin system
3. **Extended Integrations**
   - Native support for cloud storage (S3, Azure Blob, GCS)
   - Webhook notifications for job completion
   - Integration with MLOps platforms (MLflow, Weights & Biases)
4. **SDK Enhancements**
   - Support for additional languages (JavaScript/TypeScript, Go)
   - Async/await patterns for non-blocking operations
   - CLI tool built on SDK for power users
5. **Platform Features**
   - Team collaboration and dataset sharing
   - Usage analytics and cost optimization recommendations
   - Scheduled/recurring dataset updates
   - Dataset versioning and diff tools

15 References

- **PixCrawler repository:** github.com/alaamer12/PixCrawler
- **FastAPI documentation:** fastapi.tiangolo.com
- **Celery documentation:** docs.celeryq.dev
- **Azure App Service:** azure.microsoft.com/services/app-service
- **Azure Monitor:** azure.microsoft.com/services/monitor
- **icrawler documentation:** icrawler.readthedocs.io
- **MyPy static analysis guidelines:** mypy.readthedocs.io
- **Ruff linter documentation:** docs.astral.sh/ruff